

Current Architecture

Overview

The current backend is a monolithic application with multiple product surfaces inside one runtime:

- mobile API
- web routes
- Yenka store integration
- blog pages
- Socket.IO server
- cron-driven workers

Current composition

Bootstrap

- ``server.js`` loads env, configures middleware, creates the HTTP server, attaches Socket.IO, mounts routes, starts MongoDB, seeds permissions, and starts schedulers.

Runtime layers

- ``routes/``: REST entry points grouped by feature
- ``models/``: Mongoose schemas for social, content, moderation, economy, and analytics
- ``services/``: shared business logic such as rewards, notifications, privacy, and ranking
- ``middleware/``: JWT auth, RBAC helpers, community creation rules
- ``utils/``: Agora token generation, OneSignal, Cloudinary URL optimization, security audit helpers

Shared global hooks

- ``global.io``
- ``global.emitToLiveRoomForStream``
- ``global.clearLiveParticipantsForStream``

These globals let REST routes interact with realtime state without a formal event bus.

Current strengths

- feature velocity is high because most modules live in one codebase
- route surfaces are explicit and easy to trace
- data model is wide enough to support feeds, chat, livestream, verification, and rewards

Current weaknesses

- ``server.js`` is oversized and owns too many responsibilities
- route handlers sometimes contain orchestration, policy, and persistence in one file
- a number of domains mix legacy fields and newer normalized fields
- background jobs run in the same process as user traffic

Dependency hotspots

- auth middleware enriches ``req.user``, which many routes rely on implicitly
- permissions are interpreted by both ``middleware/permissions.js`` and ``models/permissions.model.js``

- notifications depend on user preferences, privacy rules, OneSignal, and Socket.IO
- feeds depend on communities, follows, privacy filtering, ads, and post approval state
- livestream depends on role rules, Agora token generation, Socket.IO rooms, and in-memory timers

Recommended improvements

1. move shared orchestration into service modules with narrower contracts
2. reduce route-side side effects on read endpoints
3. replace globals with a formal realtime/event abstraction
4. separate static web/store concerns from the social API service